Internal · not for submission

BCSE307L · COMPILER DESIGN · TEAM 2 · A15

PYTHIA Viva Guide

Everything the four of us need to explain the project in plain words, point at our own code, and answer the hard questions.

1 PYTHIA in 60 seconds
2 The whole flow, step by step
3 What changed since Review 2
4 Who did what, and how it fits
5 “Open the codebase” cheat sheet
6 Tools we chose, and why
7 Limitations and comebacks
8 The value this adds
9 Likely viva questions
10 Live demo script and backup plan
11 Glossary

Vishal Vivek 24BCE2377 · Rujuta Mangesh Kulkarni 24BDS0459 · Koliparthy Venkata Jahnavi 24BDS0132 · Aaryan Gupta 24BDS0134

Repository commit `95bbb1b` · measured on 2026-10-24 · every number here is from a command we ran (report, Appendix F)

1 PYTHIA in 60 seconds

> PYTHIA speeds up the slow parts of your Python project, and it **proves**, using your own program's calls, that the fast version gives the same answers as the original.

The problem

Python is easy to write and slow to run. Loops that add numbers, compare strings or fill tables can be 10 to 100 times slower than the same loop in C++.

People fix this in four ways: rewrite the code in C++, learn a special dialect (Cython), use a JIT (Numba, PyPy), or ask ChatGPT to translate it. Every one of these either costs a second codebase or gives you *no evidence* that the fast version behaves like the old one.

What we built

Two things, one inside the other.

- **PYTHIA, the compiler.** It reads a typed subset of Python (we call it **PySub**) and writes readable C++17. This is what you built in Reviews 1 and 2.
- **PYTHIA Studio, the app.** You give it a snippet, a file, a folder or a GitHub link. It finds the slow functions, works out their types, converts them, compiles them, checks them against real Python, and swaps them in. You see every step and every diff.

What is special

Verified

Each converted function is run against CPython on the calls your program really made. Different answer? It is thrown away automatically.

Fidelity Report

A list of every spot where Python and C++ could behave differently, with the line number and what we did about it.

Whole project

Point it at a repo. It scans, ranks, converts what it can, and tells you in plain English why it left the rest.

No LLM needed

Types are worked out by our own code. Same input, same output, free, works offline.

Readable

The C++ is something a person can open and read. The app shows it next to your Python.

SAY IT **One-line pitch.** “PYTHIA is a compiler plus an app that makes the slow parts of a Python project faster, and proves on your own program’s calls that the fast version is correct.”

The 60-second version, out loud

Python is slow on loop-heavy code. The usual fixes cost you a rewrite, or they ask you to trust a black box. PYTHIA takes an ordinary Python project, runs it once to see where the time goes, converts the hot functions to C++17 using our own type inference, and then replays the function’s real calls through both versions. If even one answer differs, the conversion is rejected. What survives is spliced in behind a one-line decorator, so the original code is still there. On our benchmark kernels the C++ is a median of 11 times faster, and every output matches CPython. It is a compiler, so we built every phase: lexer, parser, type checker, lowering, optimiser and emitter. Where Python and C++ can differ, PYTHIA says so, line by line.

Numbers to remember

78/78	valid programs where compiled output equals CPython's (the oracle)
88/88	invalid programs correctly rejected, each with the right message
91/91	everyday untyped programs converted *without any LLM* and matching CPython
21/22	LeetCode-style snippets (one is refused on purpose: `coin_change`)
14	divergence classes (D1 to D14), 1280 positioned notes across the corpus
11x	median speedup on our ten fixed kernels (nine kernels 1.5x to 29.7x; one dictionary kernel is about 29x *slower*)
11.6x	whole-program speedup on the demo repository (`dayyass/project-euler`, problem 14), output identical

2 The whole flow, one function at a time

We follow one real function, `collatz_sequence_len` from `dayyass/project-euler`, through every stage. Everything below is real output from the app, not a mock-up.

1 Your Python function

This is the function exactly as it sits in `solutions/utils.py`. It has type annotations or not; PYTHIA does not care yet.

Why: Everything starts from your own code. Nothing is rewritten by hand.

```
def collatz_sequence_len(n: int) → int:
    """
    Compute len of Collatz sequence.

    - O(1) space-complexity
    """
    result = 1
    while n ≠ 1:
        if n % 2 == 0:
            n //= 2
        else:
            n = 3 * n + 1
        result += 1
    return result
```

2 The tracer watches the program run

PYTHIA runs your entry command (`python solutions/problem_014.py`) once with a tracer on. For this function it saw **999,999 calls**, about **84.8%** of the run time, and argument types `['int']` returning `int`.

Why: Real calls give two things: which functions are worth compiling, and the types they really receive.

```
{"calls": 999999, "self_s": 9.6259183, "arg_types": ["int"], "ret_type": "int", "samples": 500}
```

3 Types are filled in (PySub)

The compiler only accepts PySub, where every variable has a type. If the function was missing annotations, PYTHIA adds them from the observed types and from how the values are used, then asks the real type checker to accept the result. Here the function already type-checked as written, so nothing had to change.

Why: No guessing without a check: the compiler is the judge.

```
def collatz_sequence_len(n: int) → int:




    result = 1
    while n ≠ 1:
        if n % 2 == 0:
            n //= 2
```

4 Tokens: the words of the program

The lexer cuts the text into tokens (keywords, names, numbers, operators). Python uses indentation, so the lexer also invents **INDENT** and **DEDENT** tokens where the block gets deeper or shallower.

Why: After this, the parser does not need to count spaces: blocks look like braces.

```
KEYWORD(def)  NAME(collatz_sequence_len)  OP(()  NAME(n)  OP(:)  NAME(int)  OP())  OP(→)  NAME(int)  OP(:)  NEWLINE
INDENT  NAME(result)  OP(=)  INT(1)  NEWLINE  KEYWORD(while)  NAME(n)  OP(≠)  INT(1)  OP(:)  NEWLINE  INDENT  KEYWO
RD(if)  NAME(n)  OP(%)
```

5 AST: the shape of the program

The parser builds a tree: a function contains statements, a loop contains a body, an expression contains operands. Every node remembers its line and column.

Why: Trees are easy to check and rewrite; error messages use the positions.

```
FuncDef 'collatz_sequence_len'
  Assign
    Name 'result'
    IntLit 1
  While
    Compare '≠'
      Name 'n'
      IntLit 1
    If
      Compare '=='
        BinOp '%'
        IntLit 0
      AugAssign '//'
        Name 'n'
        IntLit 2
      Assign
```

6 Type checking: every node gets a type

The type checker walks the tree and writes a type on each node. It pushes expected types inward (bidirectional) and widens bool to int to float where needed. Any mismatch stops here with a line and column.

Why: C++ needs to know the exact type of everything; this is where we find out.

```
FuncDef 'collatz_sequence_len'
  Assign  : int
    Name 'result'  : int
    IntLit 1  : int
  While
    Compare '≠'  : bool
      Name 'n'  : int
      IntLit 1  : int
    If
      Compare '=='  : bool
        BinOp '%'  : int
        IntLit 0  : int
      AugAssign '//'  : int
        Name 'n'  : int
        IntLit 2  : int
      Assign  : int
```

7 Lowering: make hidden things explicit (PIR)

Python converts silently (an int becomes a float in `a / b`); C++ must be told. Lowering inserts explicit conversion nodes (none in this function) and attaches a **fidelity note** to every place where C++ could behave differently. This function has **7** notes.

Why: This is the Semantic Fidelity Report being born.

```
9:14 D3 modulo sign convention (exact)
10:13 D2 floor-division rounding direction (exact)
12:19 D1 unbounded-int vs int64 (guarded)
12:23 D1 unbounded-int vs int64 (guarded)
13:9 D1 unbounded-int vs int64 (guarded)
0:0 D7 object lifetime / reclamation (guarded)
0:0 L1 copy semantics at the Python boundary (guarded)
```

8 Guard elision: remove checks that cannot fire

Each integer `+ - *` could overflow 64 bits, so by default it is a checked call. The interval analysis proves some cannot: here **0** checks were removed and **2** kept.

Why: Fewer checks, faster code, and never a removed check that was needed.

9 C++17 code generation

The emitter walks the typed tree and writes readable C++17: typed variables, loops, calls into our runtime for Python behaviour.

Why: Readable output means you can review and export it.

```
static int64_t f_collatz_sequence_len(int64_t p_n) {
    int64_t p_result = 0;

    p_result = INT64_C(1);
    while ((((p_n) != (INT64_C(1))))) {
        if ((((py_mod_i64(p_n, INT64_C(2))) == (INT64_C(0))))) {
            p_n = py_floordiv_i64(p_n, INT64_C(2));
        } else {
            p_n = py_add_i64(py_mul_i64(INT64_C(3), p_n), INT64_C(1));
        }
        p_result = py_add_i64(p_result, INT64_C(1));
    }
    return p_result;
    return 0;   /* unreachable unless the result is Optional */
}
```

10 The pybind11 bridge

A small generated wrapper turns Python arguments into C++ values, calls the function, and turns the result back. Mutated lists are written back. Anything it cannot represent faithfully raises `FallbackError`, and the original Python runs.

Why: This is the boundary, and it costs time, which is why the ranker predicts it.

```
static py::object w_collatz_sequence_len(py::object a_n) {
    pyb::Arena _arena;
    using T0 = pyb::I64;
    T0::C c_n = T0::in(a_n.ptr());
    using TR = pyb::I64;
    TR::C _r{};
    try {
        _r = f_collatz_sequence_len(c_n);
    } catch (const pyrt::PythonException &_e) {
        pyb::raise(_e);
    }
    return pyb::steal(TR::out(_r));
}

PYBIND11_MODULE(pythia_9c9bf9bd, m) {
    m.doc() = "PYTHIA-compiled module (library mode)";
    pyrt::pyrt_set_py_minor(PY_MINOR_VERSION);
    pyb::FallbackError = PyErr_NewException("pythia_9c9bf9bd.FallbackError", PyExc_Exception, nullptr);
    m.attr("FallbackError") = py::reinterpret_borrow<py::object>(pyb::FallbackError);
    pyb::SemanticError = PyErr_NewException("pythia_9c9bf9bd.PythiaSemanticError", PyExc_RuntimeError, nullptr);
```

11 Build

g++ compiles the C++ and links a Python extension, `pythia_9c9bf9bd`. Builds are cached by content: compile took **5.501 s**.

Why: A second run of the same project reuses it.

12 Replay: the oracle on your own calls

The recorded calls are run through both the original function and the compiled one: **500 of 500** calls matched (return value, exception, changed arguments; floats bit for bit). Source of inputs: `trace`.

Why: One mismatch and the function is thrown away automatically.

13 Benchmark

Python took **0.855 ms** per call and C++ **0.040 ms**: **21.3x** faster (median over repeats). Anything under 1.1x is declined.

Why: A function that compiles is not always worth compiling.

14 Splice: one line, original kept

PYTHIA adds a decorator above the original function in the working copy. The body is untouched, so the change is tiny and reversible; if the compiled module is missing, Python runs.

Why: The project keeps working whatever happens.

```
--- a/solutions/utils.py
+++ b/solutions/utils.py
@@ -1,194 +1,200 @@
-from typing import Set, Iterable
-
-
-def is_prime(n: int) -> bool:
-    """
-    Check if the number is prime using Trial division algorithm.
-
-    - O(sqrt(n)) time-complexity
-    - O(1) space-complexity
-    """
-    if n ⩽ 1:
-        return False
-
-    for i in range(2, int(n ** 0.5) + 1):
-        if n % i == 0:
-            return False
-    return True
```

3 What changed since Review 2, and why

At Review 2 you could type a PySub program and get C++ out. It worked: 17 valid programs matched CPython, 20 invalid ones were rejected, and we had added the dictionary and set libraries and moved from C11 to C++17.

But think about who would use that. Nobody has a PySub program. They have a Python project that is a bit slow. So we asked: *what would a real developer need?*

1. **Find the slow parts.** Run the program once under a tracer and rank functions by time. (Trace and Rank stages.)
2. **Work out the types.** Real code has no annotations, so we infer them ourselves from what the tracer saw and from how values are used. (Fix stage.)
3. **Prove it still works.** Replay the real calls through both versions and compare. (Verify stage.)
4. **Put it back without breaking anything.** Add one decorator line and keep the original function; if the fast version cannot run, Python runs. (Splice stage.)
5. **Show everything.** A desktop app that displays each stage, the diff and the fidelity notes, installable with one click.

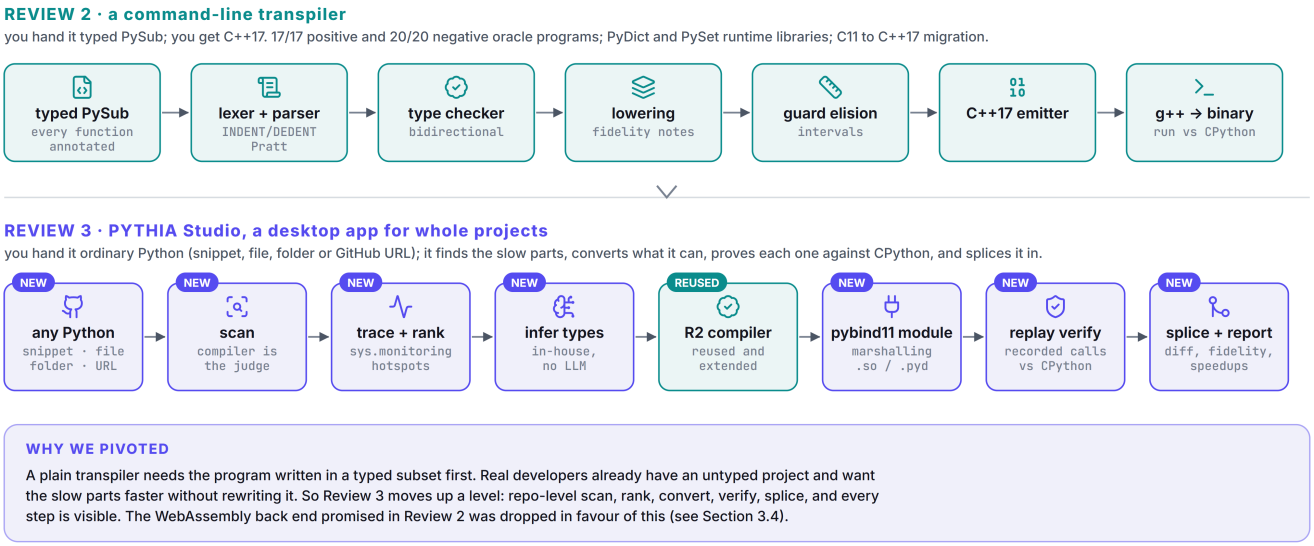


Figure 3.1. Review 2 (top) and Review 3 (bottom). “Reused” means the compiler core carried over; everything marked “new” is Review 3 work.

BE READY **We dropped the WebAssembly back end.** It was promised for Review 3. A second back end would show retargetability but would not help a developer speed up a Python project, and the time went into the Studio pipeline. Say this first, before they ask. Also say: the compiler’s IR is language-agnostic after the front end, so adding it later is an addition, not a rewrite.

Other targets that moved: the positive oracle corpus is 78, not 120 (we added an independent 91-program gate, snippets and a generator instead); memory is reclaimed per call in library mode but standalone binaries still never free memory.

4 Who did what, and how it fits

READ FIRST **What git can and cannot prove.** The first commit already holds the Review 2 code base, and the whole Studio (17 to 19 October) was committed under one name. So the split below for Reviews 1 and 2 follows the Review 1 responsibility table and the "Owner:" headers in the files; the split for the Studio work is a **proposal** that follows each person's compiler phase. Agree it as a team before the viva, and be honest if asked how the Studio was built: with AI coding assistants, recorded as co-authors on 99 of 143 commits.

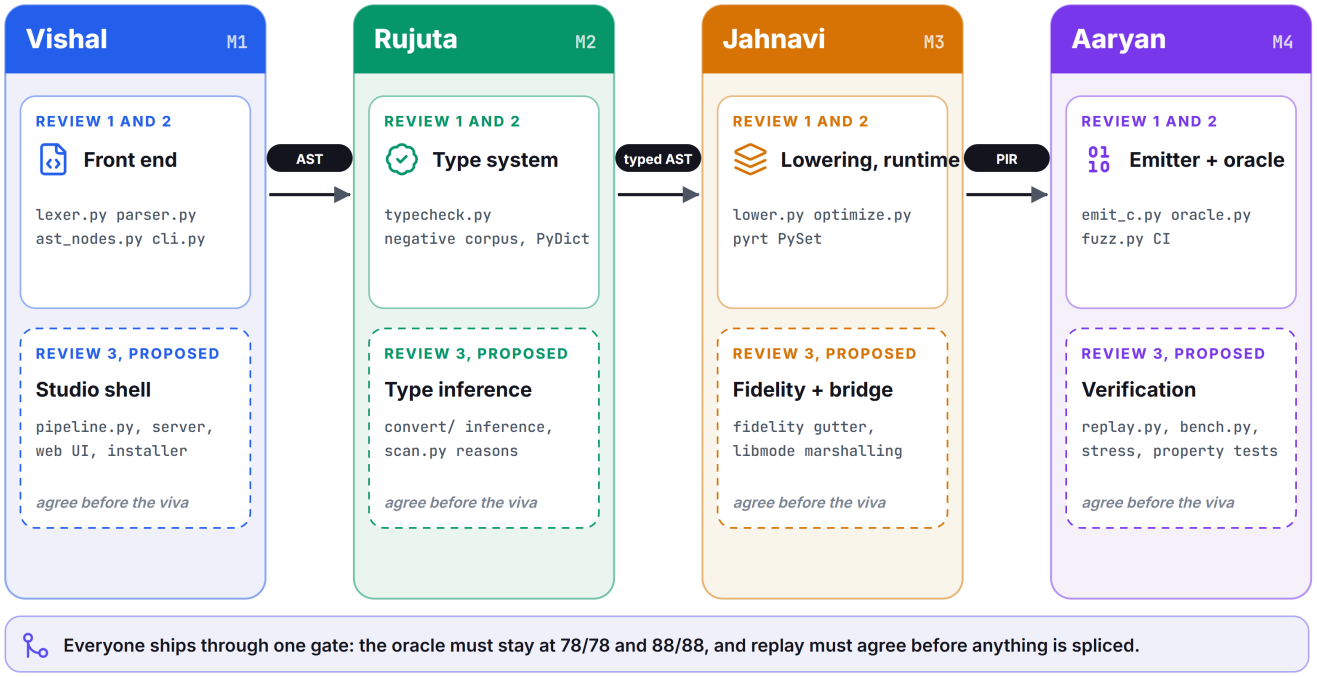


Figure 4.1. What each person hands to the next. Solid cards are Reviews 1 and 2; dashed cards are the proposed Studio split.

Vishal Vivek

24BCE2377 · team lead

WHAT THEY OWN

The **front end**: grammar, lexer, parser, AST, diagnostics; the command line, the repository and integration.

WHAT THEY BUILT

- **Lexer** with the off-side rule: turns indentation into INDENT and DEDENT tokens.
- **Parser**: recursive descent for statements, Pratt for expressions, panic-mode error recovery.
- **AST** nodes (every node has line, column and a type slot).
- Review 3: the whole **Studio** engine, server, web app, packaging, and merging parallel work.

HARDEST PROBLEM THEY SOLVED

Panic-mode recovery. The parser's `synchronize()` must always eat at least one token, or a parser parked on a bad token calls it forever. The code comment explains the bug. Also: merging eight parallel feature branches that all touched the same few dispatch functions without breaking the oracle.

HOW IT FEEDS THE NEXT PERSON

The parser hands an **AST** to the type checker. If the AST were wrong or missing positions, Rujuta's error messages would point at the wrong line.

GIT EVIDENCE

138 of 143 commits are under the owner's name, including every Studio commit and all merges. The Review 1/2 front-end files arrive in the first commit (`84f1251`), so git cannot split Review 1 work between people; ownership of those files follows the Review 1 matrix and each file's "Owner:" header.

YOU EXPLAIN THIS IN THE VIVA (PROPOSED, AGREE IT FIRST) **Studio shell.** `pipeline.py` runs the ten stages; `server/app.py` and `bus.py` carry REST calls and the WebSocket event stream; the React UI, the launcher and the Windows installer. *Explain:* how a click becomes a job, what an event is, and what Splice does to the files.

Rujuta Mangesh Kulkarni

24BDS0459

WHAT THEY OWN

The **type system**: symbol table, scopes, the bidirectional type checker, local inference, the negative test corpus, and dictionary support.

WHAT THEY BUILT

- **Type checker**: types every expression, keeps one type per variable, widens bool, int, float.
- **Negative corpus**: 88 invalid programs, each with an `expect:` marker for the message it must produce.
- **PyDict**: a Python-compatible dictionary runtime (merged as pull request #2).

HARDEST PROBLEM THEY SOLVED

Typing things that have no type yet. `xs = []` cannot be typed alone, but `xs: list[int] = []` can, because the expected type is pushed inward. That is what "bidirectional" means.

HOW IT FEEDS THE NEXT PERSON

The type checker puts a type on every AST node. Jahnavi's lowering pass reads those types to decide where a conversion or a guard is needed.

GIT EVIDENCE

Pull request #2 (merged as `079bcb5`) added `runtime/pydict.hpp` (351 lines), a 20-case C++ test and five documents. Its commits are authored as "Copilot", so say plainly that an AI assistant helped write the dictionary library. The type checker itself is in the first commit.

YOU EXPLAIN THIS IN THE VIVA (PROPOSED, AGREE IT FIRST) **In-house type inference and eligibility.** `convert/infer.py` (evidence and fixpoint), `types.py` (the lattice), `search.py` (guessing), `scan.py` and `reasons.py` (the compiler as judge, plain-English reasons). *Explain:* why a function is eligible, how types are inferred with no LLM, and why `coin_change` is refused.

Koliparthy Venkata Jahnavi

24BDS0132

WHAT THEY OWN

Lowering and the runtime: the divergence catalogue, the Semantic Fidelity Report, the `pyrt` runtime, guard elision by interval analysis, and PySet.

WHAT THEY BUILT

- **Lowering**: makes every hidden Python conversion explicit and tags each risky spot.
- **Fidelity catalogue**: D1 to D14, each with a mitigation.
- **Interval analysis** (`optimize.py`): removes overflow checks that cannot fire.
- **PySet**: Python-compatible sets (pull request #3).

HARDEST PROBLEM THEY SOLVED

Being sound. The interval pass may *keep* a check it did not need, but must never *remove* one it did. Accumulators that feed back into themselves are widened to "unknown" before the loop body is analysed, so one pass is enough.

HOW IT FEEDS THE NEXT PERSON

Lowering gives Aaryan an IR in which every conversion is explicit, so the emitter never has to guess.

GIT EVIDENCE

Pull request #3 (merged as `4ef0102`): `runtime/pyset.hpp` (533 lines) and a 366-line test, authored by Jahnavi (`ddc975b`). `lower.py`, `optimize.py` and the C runtime are in the first commit.

YOU EXPLAIN THIS IN THE VIVA (PROPOSED, AGREE IT FIRST) **Fidelity and the boundary.** The fidelity gutter and panel, `libmode.py` and `pyrt_bridge.hpp` (copy in, copy out, `FallbackError`, `OverflowError`, the arena), `rank.py` (the boundary-cost model). *Explain:* why a long list is slow to pass, what the arena frees, what D1 and D13 mean in practice.

Aaryan Gupta

24BDS0134

WHAT THEY OWN

The **back end and the safety net**: the C++ emitter, the C11 to C++17 migration, the differential oracle, the program generator, CI, the corpus and the benchmarks.

WHAT THEY BUILT

- **Emitter** (`emit_c.py`): walks the IR and writes C++17.
- **C11 to C++17 migration** (pull request #1, 4 September 2026).
- **Oracle**: runs CPython and the compiled binary, compares output byte for byte.
- **Generator and CI**: samples programs from the grammar; runs everything on every push.

HARDEST PROBLEM THEY SOLVED

Making “same behaviour” measurable. The oracle compares stdout and the exception class; the negative suite checks the *message*, not just failure. On its first run the oracle found four bugs that manual testing had missed (Review 1, section 11.4).

HOW IT FEEDS THE NEXT PERSON

Aaryan’s oracle is the gate everyone else ships through. If a change anywhere breaks equivalence, the oracle names the program.

GIT EVIDENCE

Pull request #1 (merged as `086b6f8`): commit `9264cbb` “Migrate backend from C11 to C++17”, six files, +270/−48 lines. The emitter, oracle, generator and workflow files carry “Owner: Member 4” headers and are in the first commit.

YOU EXPLAIN THIS IN THE VIVA (PROPOSED, AGREE IT FIRST) **Verification and measurement.** `replay.py` , `_replay_worker.py` , `codec.py` (the comparison), `bench.py` (median speedup, end to end), `stress.py` , `property_test.py` , the oracle and CI. *Explain:* how “verified” is decided, why floats compare bit for bit, and why the threshold is 1.1x.

5 “Open the codebase” cheat sheet

Line numbers are from commit `95bbb1b`; they were looked up by the build script, so they match the code you open. If someone edits those files, regenerate this guide.

Vishal: front end and Studio shell

Open	Point at this
`lexer.py:48`	The lexer class. The indentation stack lives in `self.indents`.
`lexer.py:79`	The main loop: layout at line start, comments, NEWLINE, strings, operators, then end-of-file DEDENTs.
`parser.py:43`	The binding-power table. One line per precedence level; `**` is the only right-associative operator.
`parser.py:512`	The Pratt loop: parse a unary, then keep folding operators while their binding power is high enough.
`parser.py:108`	Panic-mode recovery. Read the docstring: it explains the infinite-loop bug it avoids.
`cli.py:37`	The whole compiler in six lines: parse, typecheck, lower, elide guards, emit.
`pipeline.py:128`	The ten stages in order, with cancellation checks between them.
`app.py:108`	The server: REST routes plus the WebSocket that streams events.
`launcher.py:37`	How the app refuses to reuse a stale server (build fingerprint check).

30-SECOND TALK TRACK “Python marks blocks with indentation, which a grammar cannot see. My lexer keeps a stack of indent widths and turns changes into INDENT and DEDENT tokens, so after that the parser sees ordinary braces-like blocks. Statements are recursive descent; expressions use Pratt parsing with the binding-power table here, so adding an operator is one table line. When there is a syntax error the parser skips to the next statement boundary and carries on, so one run reports several errors. For Review 3 I built the pipeline that runs the ten stages and the server and app around it.”

Rujuta: type system

Open	Point at this
`typecheck.py:23`	The type representation: a name plus element, key or tuple parts.
`typecheck.py:54`	The implicit-conversion lattice: bool to int to float, plus subclass and optional rules.
`typecheck.py:71`	How two branch types are combined; `[]` next to `[1,2]` becomes `list[int]`.
`typecheck.py:82`	Parent-linked scope chain.
`typecheck.py:648`	Checking mode: the expected type is pushed inward, which is how empty lists get typed.
`typecheck.py:700`	Synthesis mode: the type is computed from the expression itself.
`n02_badarg.py:1`	A negative test: the first line says what the message must contain.
`pydict.hpp:29`	The dictionary runtime (pull request #2).

`infer.py:163`	**Studio, proposed:** the whole-program type inference engine.
`types.py:41`	**Studio, proposed:** the lattice the inference joins evidence on.
`search.py:108`	**Studio, proposed:** guessing types for parameters nobody calls.

30-SECOND TALK TRACK "The type checker has two modes. In synthesis mode it works a type out from the expression, like `1 + 2.5` being float. In checking mode it already knows what it expects, like a list annotated `list[int]`, and pushes that inward, which is the only way `[]` can be typed. Each variable keeps one type for life, which is what lets us emit unboxed C++. For Studio the same idea runs on untyped code: we collect evidence about every variable until nothing changes, and then let the real type checker accept or reject the result."

Jahnavi: lowering, fidelity, runtime, guard elision

Open	Point at this
`lower.py:26`	One fidelity note: position, class, detail, mitigation, severity.
`lower.py:40`	The catalogue of divergence classes D1 to D13 (D14 is added in two extension modules).
`lower.py:353`	Where each operator is examined and tagged.
`optimize.py:40`	The safety margin: stay inside plus or minus 2^{62}.
`optimize.py:138`	The interval analysis. `eval` computes ranges; `block` widens loop-carried variables.
`pyrt.cpp:103`	A checked addition: throws `OverflowError` instead of wrapping.
`pyrt.hpp:53`	The per-call arena that frees memory in library mode.
`pyset.hpp:39`	The set runtime (pull request #3).
`pyrt_bridge.hpp:46`	**Studio, proposed:** the boundary: copy in, copy out, write back, `FallbackError`.
`libmode.py:551`	**Studio, proposed:** generate the pybind11 wrapper and build the module.

30-SECOND TALK TRACK "Python and C++ disagree on small things that change output: floor division, the sign of modulo, integer overflow. My lowering pass finds every such spot and records a note with its position, its class and what we did. That is the Semantic Fidelity Report. Overflow checks cost time, so the interval pass proves where they cannot fire and removes them, but it is conservative: it may keep a check it did not need, never drop one it did. The runtime gives C++ Python's behaviour."

Aaryan: emitter, oracle, verification

Open	Point at this
`emit_c.py:103`	The emitter class.
`emit_c.py:176`	Walks the module and writes the translation unit.
`emit_c.py:977`	Where a guarded or unguarded `+` is chosen (look for `no_guard` just above).
`oracle.py:94`	The oracle: CPython versus the compiled binary, stdout and exception class.

`oracle.py:129`	The negative suite: the diagnostic must contain the `expect:` text.
`fuzz.py:28`	The grammar-directed program generator keeps a live type environment.
`ci.yml:1`	CI: three jobs on every push.
`replay.py:19`	**Studio, proposed:** at least 50 calls per function, topped up when too few were recorded.
`codec.py:83`	**Studio, proposed:** the comparison; floats are compared bit for bit.
`bench.py:28`	**Studio, proposed:** end-to-end timing before and after the splice, with an output hash.
`stress.py:25`	**Studio, proposed:** the stress harness over public repositories.

30-SECOND TALK TRACK “Same behaviour is a claim, so we built the thing that tries to break it. The oracle runs every program under CPython and as a compiled binary and compares the output byte for byte, plus the exception class. A second suite checks that bad programs are rejected with the right message. A generator makes new programs from the grammar so we find bugs we did not think of. In Studio the same idea runs on your own function’s recorded calls, and anything that disagrees is rejected.”

Repository tree, one line per folder

```
pythia/                     the compiler (Python)
  lexer.py  parser.py  ast_nodes.py    off-side rule, Pratt/recursive descent, AST
  typecheck.py  types_ext.py           bidirectional type checker and extensions
  classes_ext.py  tuples.py  comprehensions.py  builtins_ext.py
  lower.py  optimize.py                PIR, fidelity notes, interval guard elision
  emit_c.py (emit_cpp.py alias)  emit_ext.py  lib_ext.py   C++17 emitter
  libmode.py  classes_lib.py           pybind11 extension modules
  toolchain.py  paths.py  cli.py  driver.py  stages.py
  convert/                           in-house untyped-Python converter (no LLM)
  studio/                            Studio back end: pipeline, scan, rank, replay, splice, server/
runtime/                    C++ runtime: pyrt.hpp/.cpp, pydict.hpp, pyset.hpp, pyrt_bridge.hpp, pythia_accel/
                            (pyrt.c/.h: Review 1 C11 runtime, not used by the C++17 pipeline)
studio/web/                 React + TypeScript UI (dist/ committed)    studio/DECISIONS.md  CONTRACTS.md
packaging/windows/          installer build (build.py, launcher.c, .iss)
tests/positive  tests/negative   oracle corpora (78 and 88 programs)
tools/                      oracle.py  fuzz.py  fuzz_run.py
scripts/                    check_basics, check_snippets, property_test, stress, check_libmode, ...
demo/                       quanta (3,800 lines), textlab (480 lines), offline cache, sample run
docs/                       grammar.ebnf, Review 1 report/deck, review3/ (this report, viva guide, sources)
.github/workflows/          ci.yml (oracle, library mode, package), release.yml
```

Files added since Review 2

Everything below was added after commit `533f875` (10 September 2026, “Final Cleanup Before Review 2”).

Where	What was added
`pythia/convert/` (7 files)	The in-house untyped-Python converter: inference engine, type lattice, observer, search, preparation.

`pythia/studio/` (40 files)	The Studio back end: ingest, scan, trace, rank, fix, compile, verify, benchmark, splice, export, server, LLM fixer (optional), playground converter.
`pythia/libmode.py`, `classes_lib.py`, `lib_ext.py`, `toolchain.py`, `paths.py`	pybind11 library mode, class marshalling, toolchain discovery.
`pythia/*_ext.py`, `tuples.py`, `comprehensions.py`, `classes_ext.py`, `parse_*.py`, `driver.py`, `stages.py`	Language growth: types, classes, tuples, comprehensions, built-ins, parsing hooks.
`runtime/pyrt_bridge.hpp`, `runtime/pythia_accel/`	Python/C++ marshalling and the loader shim.
`studio/web/` (46 source files)	The React, TypeScript and Monaco application.
`packaging/windows/`	Installer assembly, launcher, Inno Setup script.
`scripts/`	check_basics, check_snippets, check_libmode, check_project, check_installed, property_test, stress, measure_coverage, install scripts, corpora.
`studio/DECISIONS.md`, `CONTRACTS.md`, `README.md`, `USER_GUIDE.md`, `INSTALL.md`, `DEMO.md`, `STATUS.md`	Design decisions, interface contracts, user and install guides, demo script, hand-over status.
`demo/`, `docs/review3/`	Two bundled demo projects with an offline cache; this review's documents and sources.

6 Tools we chose, and why

We chose	Why	Instead of
Python for the compiler	The compiler works on trees and text, not hot loops, and Studio has to import and run the user's code. Python is quick to change.	C++ or Rust: faster compiler, but slower to write and awkward to call into user code.
Hand-written lexer and parser	We needed to control the indentation rule and give precise error positions, with no extra dependency.	ANTLR, Lark, PLY: less code, but harder to get Python's layout and friendly errors.
C++17 as the target	pybind11 needs C++. Templates give typed lists. The output is readable. g++ does the optimising.	C11 (what we started with; needed GNU tricks) and LLVM IR (not readable, needs LLVM).
pybind11	Converts types and exceptions for us, one header, same compiler.	ctypes/cffi (hand-written marshalling), raw C-API (long). nanobind: we did not evaluate it.
Differential testing	Our spec is "behaves like CPython"; running both is the cheapest executable spec.	Unit tests (only what we thought of) or a formal proof (out of reach).
In-house type inference	Deterministic, free, offline. We **tried** an LLM first (Groq free tier: about 200,000 tokens a day, answers vary, needs internet) and kept it only as an optional fallback.	LLM translation.
FastAPI + WebSockets	Live event stream is a first-class feature.	Flask, Django.
React + Vite + TypeScript + Monaco in an app window	A real diff editor, runs in any browser, no second runtime to ship.	Tkinter, Qt, Electron.
Bundled MinGW-w64 GCC	A zip we can put inside the installer; the build flags target GCC.	MSVC (a separate big install).
Inno Setup	One script, per-user install, no admin rights.	WiX/MSI, NSIS.
sys.monitoring tracing	Cheap per-function events on Python 3.12+; falls back to setprofile. We never use settrace.	sys.settrace (line-level, slower).

BE CAREFUL The "instead of" column is our reasoning, not benchmarks. If asked "did you measure nanobind or Electron?", answer "no, we chose on maturity and simplicity".

7 Limitations and comebacks

Say the honest answer first, then the comeback. A panel trusts you more if you name the limit before they do.

“What about NumPy, pandas, PyTorch, OpenCV?”

HONEST ANSWER PYTHIA cannot transpile functions that call those libraries. The scanner marks them “not eligible” and says why.

CONFIDENT COMEBACK Those libraries are already C and C++ inside, so the maths is not where Python is slow. The time goes in the pure-Python code around them. PYTHIA compiles that glue and leaves the library calls in Python. Next steps: call the libraries from compiled code through the CPython C-API, map a NumPy subset onto Eigen or xtensor, or use LibTorch’s C++ API directly instead of transpiling.

“What if the code is dynamic: `eval`, monkey-patching, a variable that changes type?”

HONEST ANSWER Those are rejected with a reason, not guessed at. Our own snippet `coin_change` is refused because it returns an int on one path and a float on another.

CONFIDENT COMEBACK Refusing is the safe answer: the old version of the converter widened it to float and printed `3.0 -1.0` where Python prints `3 -1`, and we caught that. To cover more, compile one specialised version per observed type signature, or use `std::variant` for union types.

“Python ints are unbounded. What happens on overflow?”

HONEST ANSWER Compiled code uses 64-bit ints. Operations that might overflow are checked and raise `OverflowError`; an int beyond 64 bits passed into a compiled function also raises it. We measured both (2**62 * 2 raises OverflowError).

CONFIDENT COMEBACK It fails loudly instead of silently wrapping, and the fidelity report lists every site (class D1). The remedy is bignum promotion on overflow: fall back to an arbitrary-precision type only when a check fires.

“Strings and Unicode?”

HONEST ANSWER Compiled strings are ASCII bytes. A non-ASCII argument makes the wrapper raise `FallbackError`, and the original Python function runs instead, so nothing breaks, you just do not get the speedup.

CONFIDENT COMEBACK It is a safe fallback by design. The fix is UTF-8-aware strings with code-point indexing, which is a runtime change, not a compiler redesign.

“Memory: does it leak?”

HONEST ANSWER Standalone binaries never free memory (class D7). In library mode, which is what Studio uses, everything allocated during a call is freed when the call ends. We checked: memory did not grow over 200000 calls (28640 KB before, 28640 KB after), and the repo’s own check shows no growth over 3,000 calls.

CONFIDENT COMEBACK So the Studio path is safe. For standalone programs the next step is a region allocator or reference counting.

“Isn’t crossing between Python and C++ expensive?”

HONEST ANSWER Yes. Arguments are copied in and out. For a trivial function the call costs about 0.15 µs against 0.038 µs for Python’s own `len`, and a 100,000-element list costs about 1.18 ms per call to copy.

CONFIDENT COMEBACK That is exactly why the ranker exists: it predicts marshalling cost, and a function whose measured speedup is under 1.1x is “declined” and never spliced. Remedies: zero-copy buffers, keep data on the C++ side across calls, fuse neighbouring hot functions.

“Testing is not proof.”

HONEST ANSWER Correct. Verification is differential testing on recorded and generated calls. It shows agreement on those calls, not on all inputs. Untraced code paths are untested, which is why the entry command matters.

CONFIDENT COMEBACK It is evidence about *your* program, which a generic proof would not give, and anything that disagrees is rejected. Stronger forms exist and are future work: translation validation per pass, property-based fuzzing around the recorded inputs, and formally proving one pass in the style of CompCert.

“Only Python?”

HONEST ANSWER Yes: Python in, C++ out.

CONFIDENT COMEBACK Everything after the front end works on the typed tree and the IR, which do not know about Python syntax. A new front end is the only addition needed for another source language; a new emitter would give another target.

“Are dictionaries fast?”

HONEST ANSWER No. Our benchmark with a string-keyed dictionary and about 2,000 keys ran about 29 times *slower* than CPython. The cause is in `runtime/pyrt.cpp` line 131: the specialised dictionaries scan a vector for every lookup.

CONFIDENT COMEBACK Correct and slow is a fixable kind of wrong: a hash index in that one macro removes it, and the ranker would already decline such a function when the measured speedup is below 1.1x, so Studio would not splice it.

“Generators, threads, files, regex, `random`?”

HONEST ANSWER Not supported: generators are rejected, and I/O, threads, regex and unseeded randomness are refused with a reason (`explain.py`).

CONFIDENT COMEBACK Most hot numeric code does not need them. Seeded randomness is the one we would add first, because a generator with a known algorithm can be reproduced exactly.

“It takes a long time to compile.”

HONEST ANSWER The first build of a module is slow: 15.05 s cold in our library check, 5.63 s for a new module with the runtime already built, and 27 ms when the build cache hits. A whole cold project run takes minutes (Section 10.4 of the report).

CONFIDENT COMEBACK Builds are content-addressed, so a second run of the same project is fast, and the demo ships prebuilt modules. The bundled compiler means nothing is installed.

8 The value this adds

Who would use it

- **A developer with a slow Python project** who wants the hot functions faster without a rewrite, and wants to review the change as a diff.
- **A team that must justify a port.** The Fidelity Report is an audit trail: here is every place the behaviour could differ, and what we did.
- **Students and teachers of compilers.** Every phase is small enough to read and the app shows each one.

What it saves

The time to find the hot spots (the tracer does it), the time to hand-annotate (the inference does it), the time to write bindings (pybind11 glue is generated), and the time to convince yourself the fast version is right (replay does it). On the bundled TextLab demo a whole program went from 17.2 s to 6.5 s with no source edits beyond decorator lines.

Why verified matters more than raw speed

A fast wrong answer is worse than a slow right one. Tools that translate quietly change behaviour (a 64-bit integer, a different rounding rule), and the problem shows up months later. PYTHIA's promise is smaller and checkable: whatever it splices in agreed with CPython on the calls we replayed, and anything doubtful is reported or refused.

Where it could go

- **A research paper** on the divergence report as a first-class output of a source-to-source compiler, paired with replay verification. It needs a bigger independent corpus and a comparison with Cython, mypyc and Codon on the same programs.
- **A VS Code extension** that shows the gutter and diff inside the editor.
- **CI integration** that fails a build when a converted function stops matching.
- **Bignum promotion, UTF-8 strings, zero-copy buffers, type-specialised versions** (Section 7).

9 Likely viva questions, with short answers

30 questions. Learn the shape of each answer, not the words. Numbers are from this session's runs (report, Appendix F).

1 Isn't this just Cython?

No. Cython needs you to write C type declarations (or pure-Python mode annotations) and gives you a C extension with Python semantics; it does not check the result against CPython or tell you where behaviour differs. PYTHIA takes untyped code, works out the types, produces readable C++17, and checks every conversion on real calls. Cython is more complete: it handles arbitrary Python and C libraries, and we say so.

2 Why not just ask ChatGPT to convert it?

We tried an LLM first (commit `dfe6942`). Three problems: the free tier ran out (about 200,000 tokens a day), answers change between runs, and nothing proves the answer is right. PYTHIA's in-house inference is deterministic and free, and everything it produces is checked. An optional LLM fallback still exists, but its output has to pass the type checker and the replay like anything else.

3 How do you know the C++ is correct?

Two checks. The oracle compiles 78 programs and compares output and exception class with CPython byte for byte, and rejects 88 invalid programs with the right message. In Studio, every converted function is replayed on at least 50 calls (recorded, topped up with generated ones, and labelled) and the return value, exception and mutated arguments must match; floats are compared bit for bit. It is differential testing, not a proof, and we say so.

4 Why are the speedups different from Review 1?

Review 1 reported a median of 9.4x (range 3.3x to 13.8x) on 14 programs with generated C11. The oracle now reports a median of 15.2x (range 6.1x to 46.9x) on 78 programs with C++17 and `-O2 -flto`. We did not isolate each cause: the backend changed, more programs are in the set, guard elision exists, and those timings are single runs of whole processes. For performance claims we use a separate benchmark: ten fixed kernels, medians of 5 runs, median 11x.

5 What happens if someone passes NumPy code?

Functions that use NumPy are marked not eligible, with the reason "uses numpy (a C-extension library)", and keep running in Python. NumPy's own loops are already native, so the time is usually in the Python around them, which PYTHIA can compile.

6 What did *you* personally build?

Answer from Section 4 and 5: name your files, the hardest problem, and one function to open. If it is Studio work, say which part you understood and wrote or reviewed, and be straight that AI coding assistants were used and are credited in the repository history.

7 Why C++ and not LLVM?

Readable output, and pybind11 needs C++. g++ already optimises well, and we can bundle it on Windows. LLVM IR would not be readable and would need LLVM at run time. The cost: we depend on a C++ compiler, which we ship in the installer.

8 What happens on integer overflow?

Checked arithmetic raises `OverflowError` instead of wrapping (class D1). Where interval analysis proves a result stays within plus or minus 2^{62}, the check is removed. An int beyond 64 bits passed into compiled code also raises. Python would have kept going with a big integer, so this is a documented divergence.

9 Why did you drop the WebAssembly backend?

It was promised at Review 2. A second back end shows retargetability but does not help a developer speed up a Python project, and the time went to the Studio pipeline. The IR does not depend on Python syntax, so adding it later is an addition.

10 What's a DEDENT token?

Python marks blocks by indentation. When a line is indented less than the previous one, the lexer emits one DEDENT for each block that just ended, so the parser can treat blocks like braces. INDENT is the opposite. The lexer keeps a stack of indentation widths to know how many to emit.

11 Explain bidirectional type checking.

Two modes. Synthesis works a type out from the expression (`1 + 2.5` is float). Checking is given an expected type and pushes it inward (`xs: list[int] = []` types the empty list as `list[int]`). Alone, `[]` has no type; with an expected type it does.

12 Is the type system Hindley–Milner?

No. The compiler uses bidirectional local inference with one fixed type per variable. The converter for untyped code uses a whole-program fixpoint over a type lattice (join evidence until nothing changes), which is simpler than HM unification and gives up on conflicts instead of generalising.

13 Why Pratt parsing?

Expressions have many precedence levels. A Pratt parser uses one loop and a binding-power table, so adding an operator is one line, and right-associativity (`**`) is a small tweak. A cascade of grammar functions would need one function per level.

14 What is the Semantic Fidelity Report?

A list, produced by the compiler, of every place where C++ could behave differently from Python: position, class (D1 to D14), what could differ, what we did, and a severity (exact, guarded, divergent). On our corpus it has 1280 notes: 42.6% exact, 56.7% guarded, 0.7% divergent. We did not find a surveyed tool whose documentation describes a per-site report like this.

15 What is PIR?

The typed tree after lowering. It uses the same node classes, but every implicit conversion (bool to int to float) is an explicit Coerce node and every risky site carries a fidelity note, so the emitter never has to infer anything.

16 What does guard elision do, and is it sound?

Every int `+ - *` is a checked call by default. Interval analysis computes a range for each integer expression; where the range provably fits, it marks the operation unguarded. It is sound by construction: unknown means keep the check, and loop-carried variables are treated as unknown. Measured gain on our kernels is real but modest, and it does nothing for accumulators.

17 What does “verified” versus “guarded” mean?

Both mean every replayed call matched. “Guarded” also means the function contains sites we cannot prove safe, such as integer addition that could overflow int64, which raises instead of wrapping. “Verified” has none of those.

18 Why a 1.1x threshold?

Compiling a function does not make it faster if copying its arguments across the boundary costs more than it saves. We measure each function and keep it only if the median speedup is at least 1.1x. Below that it is “declined” and the number is shown.

19 What if the traced run does not cover a code path?

Then that path is untested. Replay only checks the calls the program made (plus generated inputs, labelled). That is why the entry command should exercise the hot code, and why we call it differential testing.

20 Does it change my code?

Not your folder. It works on a copy. In the copy, each accepted function gets one decorator line and the original body stays, so it is reversible, and if the compiled module cannot run, the Python body does.

21 Is it safe to run on unknown code?

No. Tracing and replay execute your project. The server listens only on 127.0.0.1 and rejects foreign Host headers, but you should only run it on code you trust.

22 How does it find the hot functions?

It runs your entry command with a tracer that uses `sys.monitoring` on Python 3.12 or later (`sys.setprofile` before that) and records calls and time per function. The ranker combines time share, eligibility and a predicted marshalling cost.

23 How does type inference work without an LLM?

Four steps: observe runtime types by running the program once; collect evidence from assignments, calls and returns into a lattice until nothing changes; for parameters nobody calls, try plausible type combinations; then let the real type checker accept or reject the whole result. If anything is ambiguous, such as int on some paths and float on others, it refuses.

24 Did you use AI to build this?

Yes, and the repository says so: Claude (Opus 5.5 and Sonnet 5.5) and GitHub Copilot appear as co-authors on 99 of 143 commits, including the dictionary library. The product itself does not need an LLM to run. We are responsible for what we submit, and you should be able to explain any file you claim.

25 What does “the compiler is the judge” mean?

The scanner does not keep a list of supported features. It extracts each function into a standalone module and runs the real parser and type checker on it. If it passes, the function is eligible. So when the compiler gains a feature, the scanner needs no change.

26 What are the time complexities?

Lexing, parsing, type checking, lowering, interval analysis and emission are linear in the size of the unit. Inference iterates passes of linear cost until the lattice stops changing (it has finite height). The search tries at most 400 type assignments, each a full check. Build time is dominated by g++.

27 Why is the end-to-end speedup smaller than the per-function speedup?

Amdahl’s law: only the converted functions get faster. On TextLab, functions are 4x to 44x faster but the whole program is 2.6x, because the rest still runs in Python and the program has other costs.

28 What are the known bugs?

Honest list: the snippet `coin_change` is refused (21 of 22 snippets); the grammar file is the v0.1 core and has not been regenerated for the later language; some fidelity-note text is stale (D8 still mentions `setjmp`); and STATUS.md lists three stress-harness cases found earlier that we did not re-investigate.

29 What would you do with another month?

Bignum promotion and UTF-8 strings, zero-copy buffers, type-specialised versions so int-or-float functions convert, a VS Code extension, and a larger independent corpus with a comparison against Cython, mypyc and Codon so the novelty claim could become a paper.

30 Can I see it work?

Yes. Section 10 is the demo script: it runs the app on a small public repository, shows the pipeline, one diff, and one function it correctly leaves alone, in under three minutes.

10 Live demo script and backup plan

Pick	Detail
Main	**dayyass/project-euler** @ `ad501f8`, command `python solutions/problem_014.py`. `collatz_sequence_len` 21.3x (500/500 replays), caller declined 0.81x, whole program 8.9 s to 0.8 s (11.6x), output identical.
Backup 1	Same repo, `python solutions/problem_010.py`: `is_prime` 1.8x, whole program 5.4x.
Backup 2	TextLab (bundled): `python main.py --small`, 9 functions accelerated.

Clicks, about 2 to 3 minutes

1. Open folder (or clone the URL), type the entry command, press Analyse.
2. Watch the stepper; at Trace say “about a million calls”; at Rank say “measured, not guessed”.
3. Click `collatz_sequence_len`: Guarded, 21.3x, 500/500 replays match. Diff: one decorator line.
4. Journey tab: walk the stages (Section 2).
5. Click `problem_014`: Declined 0.81x. “Careful, not reckless.”
6. End-to-end card: 11.6x, output identical; original folder untouched.

If something fails

1. **Replay a saved run** (no network): run `docs\review3\demo\install_saved_runs.py`, open `/?job=c45cc4b3&replay=1`.
2. **Local folder** instead of the URL.
3. **TextLab** (`python main.py --small`) or its saved run.

Before you go on stage

- ☐ Laptop charged and plugged in
- ☐ Installer rebuilt after this session’s runtime fixes (the one in `dist/` is from 19 Oct)
- ☐ No stale server on port 8765 (also check WSL)
- ☐ Saved runs installed; dry run done this morning, cold and warm
- ☐ Wi-Fi-off test passed; light theme; window at least 1440×900

BE HONEST The main demo compiles **one** function and the repo has no licence file. If asked “why this repo”: it was the best small, unknown, pure-Python project that went through the real pipeline with identical output; most small repos use classes, generators or global state, which v1 declines with a stated reason.

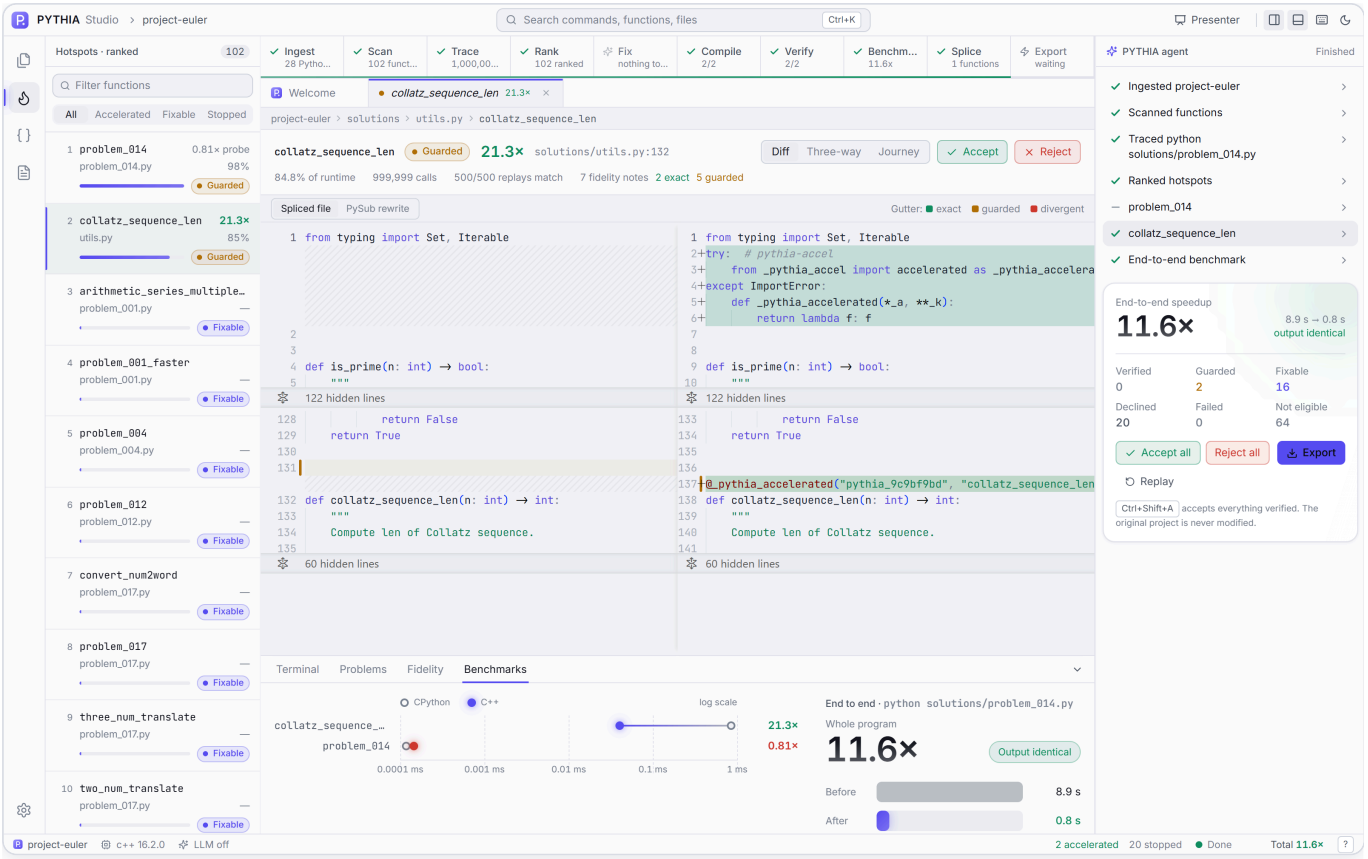


Figure 10.1. The diff for the main demo: one decorator added, original body untouched.

11 One-page glossary

Term	Meaning
AST	Abstract syntax tree: the program as a tree of statements and expressions.
Arena	Memory region freed in one go when a library call ends.
Bidirectional type checking	Synthesise a type from an expression, or check it against an expected type pushed inward.
Boundary / marshalling	Copying values between Python objects and C++ objects when a compiled function is called.
DEDENT / INDENT	Tokens the lexer invents when indentation decreases or increases.
Differential testing	Run two implementations on the same input and compare their behaviour.
Divergence (D1 to D14)	A place where C++ could behave differently from Python; each class has a mitigation.
Fidelity note / report	A positioned record of one divergence; the report is the full list.
Guard	A runtime check, for example for integer overflow.
Guard elision	Removing a guard that analysis proves can never fire.
Hotspot	A function where a program spends much of its time.
Interval analysis	Tracking a (low, high) range for each integer expression.
IR / PIR	Intermediate representation; PIR is our typed tree with explicit conversions.
Library mode	Compiling functions into a Python extension module instead of a standalone program.
Lattice	A set of types with a “join” that combines evidence; bool, int, float is a tiny one.
Off-side rule	Block structure carried by indentation.
Oracle	The test harness that compares compiled output to CPython’s.
pybind11	A C++ library that builds Python extension modules.
Pratt parsing	Expression parsing with a binding-power table.
PySub	The typed Python subset PYTHIA compiles.
Replay	Feeding recorded calls to both the original and the compiled function and comparing.
Splice	Adding a decorator so the compiled function is used, keeping the original body.
Declined	Compiled and measured, but not spliced because it was not at least 1.1x faster.
Translation validation	Checking each translation, instead of proving the translator correct.